

Linear models: recursive least squares (RLS)

Standard least squares: $f(\mathbf{x}_n) = \mathbf{x}_n^\top \mathbf{w}$, loss function $(f(\mathbf{x}_n) - y_n)^2$, covariance matrix $\Sigma_n = X^\top X$, $X \in \mathbb{R}^{n \times d}$. The solution for $f^*(\mathbf{x}) = \mathbf{x}^\top \mathbf{w}^*$ is

$$\mathbf{w}^* = (X^\top X)^{-1} X^\top y = \Sigma_n^{-1} \sum_{i=1}^n \mathbf{x}_i y_i$$

RLS: based on the matrix inversion lemma:

Initialize $\mathbf{w}_0 = 0 \in \mathbb{R}^d$, and $\Gamma_0 = I \in \mathbb{R}^{d \times d}$.

Iteration:

$$\begin{aligned}\Gamma_n &= \Gamma_{n-1} - \frac{\Gamma_{n-1} \mathbf{x}_n \mathbf{x}_n^\top \Gamma_{n-1}}{1 + \mathbf{x}_n^\top \Gamma_{n-1} \mathbf{x}_n} \\ \mathbf{w}_n &= \mathbf{w}_{n-1} - \Gamma_n \mathbf{x}_n (\mathbf{x}_n^\top \mathbf{w}_{n-1} - y_n)\end{aligned}$$

It can be shown that $\Gamma_n = \Sigma_n^{-1}$.

The regularized version is obtained as usual by introducing a regularizer in the loss function, $\sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i)^2 + \lambda \|\mathbf{w}\|_2^2$, and starting from $\Gamma_0 = (I + \lambda)^{-1}$. This converges to $\Gamma_n = (\Sigma_n + \lambda I)^{-1}$.

Explicación — Diapositiva 1

Tema: Linear models: recursive least squares (RLS)

Explicación:

En el contexto de modelos lineales, el método de **Least Squares (LS)** proporciona una solución para minimizar la suma de los cuadrados de las diferencias entre las predicciones y los valores reales. Para un modelo lineal dado por $f(x_n) = x_n^T w$, donde x_n es una muestra de entrada y w son los pesos desconocidos, la función de pérdida es $(f(x_n) - y_n)^2$. La matriz de covarianza Σ_n se define como $\Sigma_n = X^T X$, donde X es una matriz de diseño de tamaño $(n \times d)$.

El objetivo del **Recursive Least Squares (RLS)** es actualizar los pesos w de manera eficiente cuando se recibe un nuevo dato x_n y el valor correspondiente y_n . El proceso inicializa los pesos w_0 como cero y la matriz de varianza Γ_0 como la identidad I . A medida que se recogen más datos, se actualiza la matriz de varianza Γ_n utilizando la fórmula $\Gamma_n = \Sigma_n^{-1}$.

La versión regularizada del RLS introduce un término de regularización en la función de pérdida, que se expresa como $\sum_{i=1}^n (w^T x_i - y_i)^2 + \lambda \|w\|_2^2$. Aquí, λ es un parámetro de regularización que controla la penalización de los pesos. La matriz de varianza Γ_n se calcula como $\Gamma_n = (\Sigma_n + \lambda I)^{-1}$, donde I es la matriz identidad.

Una ventaja del RLS es su capacidad para actualizar los pesos de manera recursiva, lo que significa que no es necesario calcular de nuevo los pesos cada vez que se recibe un nuevo dato. Este método es especialmente útil en aplicaciones donde se requiere una actualización constante y en tiempo real de los modelos.

Stochastic Gradient Descent

Instead of updating w_n using

$$w_n = w_{n-1} - \Gamma_n x_n (x_n^\top w_{n-1} - y_n)$$

use gradient descent:

$$w_n = w_{n-1} - \alpha_n \nabla V(f(x_n), y_n)$$

where α_n is the learning rate, and $V(f(x_i), y_i)$ a convex, differentiable loss function.

The learning can be done in batches, and the order of the input samples can be stochastic or deterministic.

Explicación — Diapositiva 2

Tema: Stochastic Gradient Descent

Explicación:

Stochastic Gradient Descent (SGD) es una variante del método de descenso de gradiente utilizado en aprendizaje automático y optimización de funciones. En lugar de actualizar los pesos $\mathbf{w}_n$ utilizando la derivada global del gradiente de la función de pérdida $V(\mathbf{w}_n)$ sobre todo el conjunto de datos, SGD realiza actualizaciones iterativas más pequeñas y rápidas. Esta estrategia se basa en calcular el gradiente de la función de pérdida para cada muestra individual (x_i, y_i) en el conjunto de entrenamiento, lo que permite realizar ajustes de pesos en cada paso de entrenamiento.

El proceso de actualización de pesos en SGD se describe por la siguiente ecuación:

$$\mathbf{w}_{n+1} = \mathbf{w}_n - \alpha_n \nabla V(\mathbf{w}_n; x_i, y_i)$$

donde α_n representa la tasa de aprendizaje o learning rate, que es un parámetro crucial que controla la magnitud de las actualizaciones de pesos. La tasa de aprendizaje debe ser adecuada para evitar oscilaciones innecesarias y para asegurar que el algoritmo converge hacia un mínimo local o global.

Una ventaja de SGD es que puede manejar grandes conjuntos de datos de manera eficiente, ya que no requiere procesar todos los datos en cada iteración. Esto se logra mediante la división del conjunto de datos en "batches" o "máquinas", donde el algoritmo itera sobre estos grupos de muestras en lugar de considerar todas las muestras al mismo tiempo.

Además, SGD puede funcionar bien incluso cuando la función de pérdida no es convexa, aunque en este caso puede ser menos eficiente que otros métodos de optimización. Sin embargo, si la función de pérdida es convexa, como sucede comúnmente en muchos problemas de aprendizaje supervisado, SGD puede converger rápidamente hacia un mínimo local óptimo.

En cuanto al orden de las muestras en el conjunto de entrenamiento, SGD puede realizar ajust

Kernel methods: KRR (aka LS-SVM)

In kernel methods, the predictor at the n -th step has the form

$$f_n(\mathbf{x}) = \langle \mathbf{w}_n, \mathbf{x} \rangle = \sum_{k=1}^n (\alpha_n)_k \langle \mathbf{x}_k, \mathbf{x} \rangle = \sum_{k=1}^n (\alpha_n)_k K(\mathbf{x}_k, \mathbf{x})$$

where $\mathbf{w}_n = X_n^\top \alpha_n$, and $\alpha_n = (K(\mathbf{x}_n, \mathbf{x}_n) + \lambda I)^{-1} \mathbf{y}_n$.

Naive kernel regression: weights are recursively obtained using

$$\begin{aligned}\alpha_o &= 0 \\ (\alpha_n)_i &= (\alpha_{n-1})_i, \quad i = 1, \dots, n-1 \\ (\alpha_n)_n &= \gamma_n \left(y_n - \sum_{k=1}^{n-1} (\alpha_{n-1})_k K(\mathbf{x}_k, \mathbf{x}_n) \right)\end{aligned}$$

begin γ_n is the learning rate.

Explicación — Diapositiva 3

Tema: Kernel methods: KRR (aka LS-SVM)

Explicación:

En el contexto de los métodos de kernel, la regresión por kernel (KRR) es una técnica que permite predecir valores de salida basándose en características no linealmente relacionadas con las entradas de entrada. Este método se utiliza comúnmente en el marco de los sistemas de aprendizaje supervisado, donde la tarea es estimar una función de regresión que minimice la diferencia entre los valores reales y los predichos.

El predictor en el paso n -ésimo de la regresión por kernel tiene la siguiente forma: $\hat{w}_n = X_n^{\text{top}} \alpha_n$, donde α_n es un vector de pesos que se obtiene recursivamente a partir de la siguiente ecuación: $\alpha_n = (K(x_n, x_n) + \lambda I)^{-1} y_n$. Aquí, $K(x_n, x_n)$ representa la matriz de similitud o kernel, que mide la similitud entre el punto de entrada x_n y sí mismo, y y_n son los valores de salida correspondientes a los puntos de entrada. El término λI es una regularización que ayuda a evitar el sobreajuste del modelo.

Es importante notar que la regresión por kernel puede ser vista como una generalización de la regresión lineal. Sin embargo, en lugar de utilizar pesos constantes para cada entrada, KRR utiliza pesos que dependen de la estructura del espacio de características generado por el kernel. Esto permite a KRR capturar relaciones no lineales entre las entradas y las salidas, lo cual es particularmente útil cuando estas relaciones son complejas y no pueden ser descritas por una función lineal.

Además, KRR es conocido por su capacidad para manejar grandes conjuntos de datos sin necesidad de almacenar todos los datos originales. Esto se debe a que la matriz de similitud K se calcula solo para los puntos de entrada relevantes, lo que reduce significativamente el tamaño del problema original.

Por otro lado, la regresión por kernel también tiene sus limitaciones. Como menciona el audio, estos métodos